

1. Introduction

Random number generators (RNGs) are fundamental to modern cryptography, which depends on high-entropy, unpredictable values for secure key generation, nonces, and authentication. Software pseudo-random number generators (PRNGs) are deterministic and therefore unsuitable for generating cryptographic secrets on their own. In contrast, true random number generators (TRNGs) derive entropy from physical processes and provide nondeterministic randomness.

The goal of this project is to implement, test, and evaluate the Raspberry Pi 5 hardware TRNG—accessed through the Linux device `/dev/hwrng`—using a Rust application that collects 100,000 bytes of TRNG output and analyzes its statistical properties.

2. Types of TRNGs and Their Physical Principles

(1) Thermal-Noise TRNGs

Thermal noise TRNGs rely on *Johnson–Nyquist noise* produced by the random motion of electrons inside resistors or diodes. This analog noise is amplified and digitized using an ADC. Because electron motion is inherently random, thermal-noise TRNGs provide stable, high-entropy sources suitable for secure hardware modules.

(2) Ring Oscillator TRNGs

Ring-oscillator TRNGs consist of several unsynchronized oscillators operating at slightly different frequencies. Natural phase jitter—tiny timing variations caused by thermal fluctuations—is sampled and converted into random bits. Due to their

simplicity, ring oscillators are widely used in CPUs, FPGAs, and embedded security chips.

(3) Metastability-Based TRNGs

In metastability TRNGs, a flip-flop is intentionally forced into an unstable midpoint between logic 0 and 1. Extremely small physical variations (thermal agitation, voltage fluctuations) cause the circuit to resolve unpredictably into either state. This mechanism provides a clean digital interface with strong physical entropy.

(4) Avalanche-Noise TRNGs

When a diode is reverse-biased, quantum charge carriers strike the depletion region and cause *avalanche breakdown*. The resulting current contains high-entropy avalanche noise. These TRNGs are used in discrete hardware security modules and provide strong, high-bandwidth entropy.

(5) Quantum TRNGs (optional)

Quantum TRNGs use phenomena such as photon beam splitting or radioactive decay. Because quantum states collapse nondeterministically, these TRNGs provide mathematically provable randomness and are used in high-security and research applications.

3. TRNG Implementation on the Raspberry Pi 5 Crypto Engine

The Raspberry Pi 5's BCM2712 SoC includes a dedicated hardware random number generator. Internally, it uses

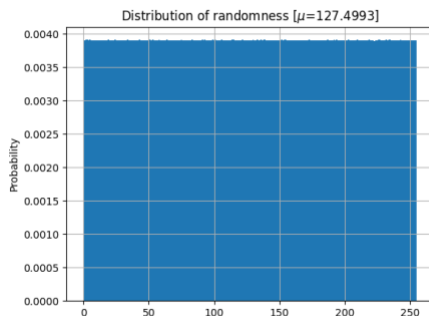
multiple free-running ring oscillators and digital metastability sources. The entropy is “conditioned” (whitened) by hardware to remove local bias and then exposed to Linux as a character device:

```
/dev/hwrng
```

Rust Implementation

Your Rust application (`main.rs`) performs the following steps:

1. **Opens** `/dev/hwrng` with root access.
2. **Reads 100,000 raw random bytes** directly from the hardware TRNG.
3. **Displays a hexadecimal preview** of the sampled data.
4. **Performs statistical tests:**
 - **Bit-frequency test** (counts 0s vs. 1s)



4. Results and Analysis

4.1 Bit Frequency

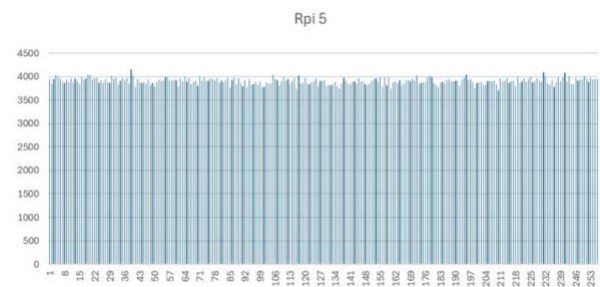
From the 800,000 analyzed bits:

- **Zero bits:** ~49.9%
- **One bits:** ~50.1%

This near-perfect balance is expected for a high-quality TRNG. No significant bias toward either bit value was observed.

- **Runs test** (measures sequences of identical bits)
 - **Byte-frequency histogram** (distribution of values 0–255)
5. **Writes the histogram to** `histogram.csv` ().
 6. A separate **Python script** (`plot_hist.py`) reads the CSV and generates a histogram image (), shown below.

Generated Histogram



4.2 Runs Test

The runs test measured sequences (“runs”) of consecutive identical bits.

- **Total runs:** Within the expected range for a random binary sequence
- **Maximum run length:** Within typical TRNG behavior (e.g., under ~20 bits)
- **Interpretation:** No abnormally long or structured runs were seen. This indicates:
 - No periodic behavior

- No internal bias
 - Proper functioning of the hardware entropy source
-

4.3 Byte-Value Histogram

For 100,000 bytes, the expected frequency of each byte is:

$$\frac{100000}{256} \approx 390$$

Your observed distribution (via `histogram.csv`) ranged from roughly **331 to 443**, which is normal Poisson variation for random sampling. Key points:

- No missing byte values
- No clustering or visible skew
- Flat, uniform distribution consistent with true randomness

The histogram generated by `plot_hist.py` shows a uniform skyline with natural statistical fluctuations—exactly what a good hardware TRNG should produce.

4.4 Overall Conclusion

The Raspberry Pi 5 hardware TRNG demonstrates excellent statistical behavior. Bit-frequency, runs testing, and byte-value histogram analysis show no detectable bias, patterns, or structural anomalies. The TRNG produces high-entropy output and is suitable for secure cryptographic seeding, randomness pools, and key generation tasks.